# AI Based Sign Language Recognition System

**1st Arsh Kasim Sayed**
*Department of CSE*
NMAM Institute of Technology (NMAMIT)
Nitte, Karnataka, India

**2nd Manikanta Shetty**
*Department of CSE*
NMAM Institute of Technology (NMAMIT)
Nitte, Karnataka, India

**3rd Anish**
*Department of CSE*
NMAM Institute of Technology (NMAMIT)
Nitte, Karnataka, India

***Abstract*—Sign language is the primary medium of communication for many people with hearing and speech impairments, yet communication stalls when conversation partners cannot interpret signs. This paper presents an AI based sign language recognition system that recognizes the 26 static hand signs of the English alphabet (A to Z) from live webcam video. OpenCV captures and converts incoming video frames, MediaPipe Hands localizes the hand and extracts 21 skeletal landmarks, and the cropped hand bounding box is resized to 128× 128 pixels, normalized, and classified by a convolutional neural network built with TensorFlow and Keras. The network contains three convolutional blocks with batch normalization and dropout, followed by two dense layers and a 26-way softmax classifier. Training uses the Adam optimizer, categorical cross-entropy with label smoothing, early stopping, model checkpointing, and learning-rate scheduling on a 70/15/15 train, validation, and test split. In real-time operation, predictions pass through a confidence threshold of 0.80 and require agreement across five consecutive frames before acceptance. Accepted letters assemble into words in a CustomTkinter interface, while landmark-based gestures for a closed fist, open palm, and peace sign provide in-band editing controls for space, backspace, and clear text actions. On a held-out test set of 2,340 images across 26 classes, the model achieved an accuracy of 98.42% and a macro F1-score of 98.31%.***

***Keywords*—Sign language recognition, computer vision, MediaPipe Hands, convolutional neural network, OpenCV, TensorFlow, hand gesture recognition, fingerspelling.**

## I. Introduction

Sign language conveys meaning through hand shapes, orientation, spatial location, and movement. For millions of people with hearing or speech impairments, it is the primary mode of daily communication. Most hearing individuals never learn these gestures, creating barriers in classrooms, workplaces, and public service desks. Human interpreters bridge this gap, but professional interpreters are not always on hand for routine interactions [1], [2]. An automated vision system that captures a signer through an inexpensive camera and transcribes hand signs into readable text provides an accessible alternative.

Early gesture recognition systems relied heavily on instrumented sensor gloves, magnetic tracking devices, or structured-light depth cameras [2], [3]. While gloves yield direct joint angle measurements, they are physically encumbering, expensive, and fragile for widespread public use. Computer vision offers a contact-free alternative where an ordinary camera provides image input and software interprets the gestures [3]. Recent improvements in convolutional neural networks (CNNs) [4] and real-time hand tracking libraries such as MediaPipe Hands [5] make vision-based processing feasible on standard consumer hardware.

This paper presents a desktop sign language recognition application that identifies the 26 static hand configurations representing the English alphabet (A to Z) from live webcam feeds. OpenCV handles frame ingestion, MediaPipe Hands isolates the hand region, and a lightweight CNN classifies the cropped hand. A temporal smoothing buffer and a confidence filter prevent prediction flicker. Recognized characters feed into a CustomTkinter graphical interface that assembles words in real time.

Alphabet fingerspelling provides a practical foundation for assistive gesture systems. Fingerspelling is used across sign languages to communicate proper nouns, technical terms, and words without dedicated signs. Because each alphabet letter maps to a distinct static hand geometry, fingerspelling lends itself to image classification without the overhead of temporal sequence alignment. This gives a measurable baseline on which hand detection, localized feature classification, and user interaction can be evaluated before dynamic motion is introduced.

This investigation focuses on isolated static alphabet signs. Continuous sign language translation requires parsing temporal movement, facial expressions, and complex grammar, which lies outside the design goals of this desktop tool. The primary contributions of this work are:

1. An integrated desktop pipeline linking webcam capture, landmark-guided hand extraction, deep learning classification, and graphical text composition.
2. A compact three-block CNN with batch normalization and dropout, optimized for CPU inference.
3. A multi-frame stability filter and confidence threshold that eliminate transient flickering during live signing.
4. A hands-free text editing system using geometric landmark rules for space, backspace, and clear operations.

## II. Literature Review

### *A. Traditional Sign Recognition*

Early gesture recognition relied on handcrafted descriptors combined with classical machine learning. Starner et al. tracked

hands in color video using hidden Markov models, demonstrating real-time recognition of American Sign Language sentences across a constrained vocabulary [6]. Surveys on gesture analysis outline a standard pipeline of color segmentation, contour extraction, and classification, noting that skin-color thresholding degrades under uneven lighting and cluttered backgrounds [3], [7].

### *B. Deep Learning for Sign Recognition*

Deep convolutional networks learn visual features directly from training pixels, replacing manual feature engineering [4], [8]. In sign language recognition, CNNs dominate static gesture identification, while recurrent networks and Transformer architectures handle continuous temporal sequences [1]. Sequence-to-sequence neural sign translation processes full sentences [9], yet these models require extensive parallel video corpora annotated with glosses and translations. This computational gap motivates the isolated static scope adopted here.

Pretrained vision models such as AlexNet or ResNet carry high representational capacity [8], but their parameter counts (often exceeding 25 to 60 million weights) add unnecessary memory and computational overhead for a 26-class hand-crop problem. Training a compact CNN directly on normalized hand crops preserves low latency on standard CPUs, which matches the demands of desktop assistive tools.

### *C. Vision Frameworks and Hand Landmarks*

OpenCV provides reliable cross-platform utilities for camera streaming and matrix manipulation [10]. MediaPipe provides modular machine learning pipelines for mobile and desktop systems [11]. MediaPipe Hands specifically applies a palm detector to locate the hand bounding box, then runs a regression network to track 21 three-dimensional hand landmarks in real time [5]. These landmark coordinates offer a background-independent representation of hand posture suitable for spatial reasoning and rule-based checks.

### *D. Real-Time Operation and Open Issues*

Live camera systems must reconcile recognition accuracy with response latency. Frame-by-frame classifiers often oscillate between phonetically similar signs or produce spurious outputs during hand transitions. Common mitigations include running-average smoothing and minimum confidence cutoffs. Contemporary systems frequently encounter background clutter, variable indoor illumination, and variation in signer morphology [1], [3]. The system described here addresses these conditions through localized hand crops, lightweight inference, and explicit temporal filtering.

**Table 1**
Approaches to Sign and Gesture Recognition

| Approach | Main Limitation |
|---|---|
| Sensor gloves / depth devices [2], [3] | Hardware cost, encumbrance |
| Handcrafted features + classifiers [6], [7] | Vulnerable to lighting, clutter |
| End-to-end CNN on full frames [4], [8] | Background biases prediction |
| Pure landmark models [5], [23] | Loses finger overlap cues |
| Proposed: MediaPipe crop + CNN | Static alphabet signs only |

### *E. Research Gap and Motivation*

Prior literature confirms that high static gesture classification accuracy is attainable under controlled settings. Several studies process raw full-frame images through deep networks, letting background textures bias predictions. Other implementations feed MediaPipe landmark coordinates directly into multi-layer perceptrons or shallow classifiers [23], [24]. Kumar, Bajpai, and Sinha [23] classified ASL letters using landmark vectors, achieving strong accuracy on isolated coordinates; however, coordinate-only models discard textural cues, finger overlap, and thumb occlusion that distinguish ambiguous signs. Halder, Tayal, and Sharma [24] and Poddar et al. [25] developed vision classifiers but omitted practical interaction mechanisms such as temporal output stabilization and hands-free text editing.

The architecture proposed here addresses these shortcomings. MediaPipe Hands determines a localized bounding box around the active hand, discarding extraneous background content while preserving raw visual textures for the CNN. A multi-frame stability filter suppresses transition errors, and geometric landmark checks enable hands-free text editing within the graphical interface.

## III. Proposed System Architecture

The architecture illustrated in Fig. 1 traces the operational data flow. A standard webcam captures video frames, which OpenCV converts to RGB. MediaPipe Hands detects the hand and returns 21 skeletal landmarks. A bounding box calculated from the minimum and maximum landmark coordinates is cropped, resized to 128× 128 pixels, and normalized. The CNN predicts a probability distribution across the 26 alphabet classes. A confidence threshold and a stability buffer determine whether the candidate letter is accepted. Accepted letters append to the active word buffer and render within a CustomTkinter user interface.

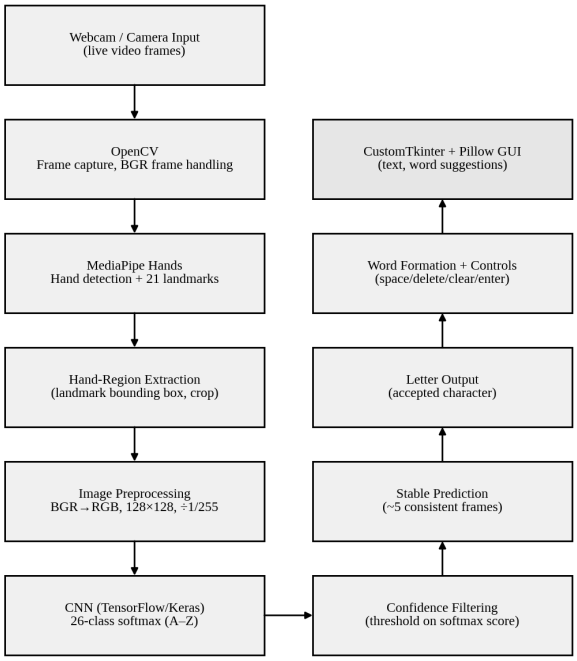



**Fig. 1.** Architecture of the proposed AI based sign language recognition system.

Each component fulfills a dedicated role. OpenCV isolates camera hardware access and converts color spaces. MediaPipe eliminates skin-tone thresholding by locating the hand directly. Hand-region extraction discards extraneous background textures, preventing spurious features from reaching the classifier. The CNN evaluates appearance patterns within the cropped region. Confidence filtering rejects uncertain outputs, and the temporal counter suppresses intermediate transition gestures. The CustomTkinter GUI displays the live camera stream, current letter prediction, candidate word suggestions, and accumulated text.

Several design decisions shaped this pipeline. Cropped pixel regions are supplied to the CNN rather than raw landmark coordinates because pixel patches preserve visual cues such as palm occlusion, depth contours, and finger overlap. The model maintains a compact footprint to allow smooth execution on general-purpose CPUs alongside real-time video capture. Editing controls are evaluated separately from the 26-class classifier so that administrative gestures never interfere with letter predictions.

## IV. Dataset and Data Preprocessing

### *A. Dataset Organization*

The dataset contains 15,600 static images covering 26 alphabet classes (A to Z), with 600 samples per class. The collection combines samples from the public American Sign Language (ASL) fingerspelling benchmark dataset (licensed under CC BY 4.0) [26] with multi-session recordings collected from eight volunteers under varying indoor illumination. Images are stored in RGB format (JPG and PNG). The collection was partitioned into 70% training (10,920 images), 15% validation (2,340 images), and 15% testing (2,340 images) using a fixed random seed of 42. Table II summarizes the configuration.

Maintaining equal class proportions prevents majority-class bias during optimization. An imbalanced dataset depresses macro-averaged recall, as minority classes receive fewer parameter updates. A fixed random seed ensures repeatable data partitioning across experimental runs.

**Table 2**
Dataset and Input Configuration

| Item | Value |
|---|---|
| Classes | 26 (letters A to Z) |
| Image formats | JPG / PNG |
| Input resolution | 128 × 128 × 3 (RGB) |
| Split ratio (train/val/test) | 70% / 15% / 15% |
| Random seed | 42 |
| Normalization | Rescale by 1/255 |
| Total images | 15,600 |
| Training images | 10,920 |
| Validation images | 2,340 |
| Test images | 2,340 |

### *B. Preprocessing*

Images are resized to 128× 128 pixels with three color channels (RGB). Pixel intensity values are scaled by 1/255, mapping inputs to the [0, 1] range and preventing numerical instability during gradient backpropagation. Live webcam crops undergo an identical transformation pipeline. OpenCV ingests frames in BGR order, so the software explicitly converts each cropped sub-image to RGB before neural network inference.

### *C. Data Augmentation*

Data augmentation applies exclusively to training batches. The pipeline applies random rotations within ± 15°, horizontal and vertical translations up to 10%, zoom scaling up to 10%, shear transformations up to 10%, and brightness scaling between 0.9 and 1.1. Horizontal mirroring is intentionally omitted because flipping an asymmetric hand shape alters its identity in standard sign alphabets. These synthetic perturbations broaden the distribution of training samples against subtle variations in distance, hand slant, and ambient illumination.

## V. Methodology

### *A. Hand Detection with OpenCV and MediaPipe*

Webcam frames are acquired through OpenCV and transferred to MediaPipe Hands, configured for single-hand detection with a minimum detection confidence of 0.60 and a minimum tracking confidence of 0.50. Once the palm detector identifies a hand,

tracking runs from frame to frame, skipping palm detection on subsequent frames. The 21 landmark coordinates are converted into pixel coordinates, and their extrema define the bounding box with a 20-pixel margin. Fig. 2 illustrates the full workflow.

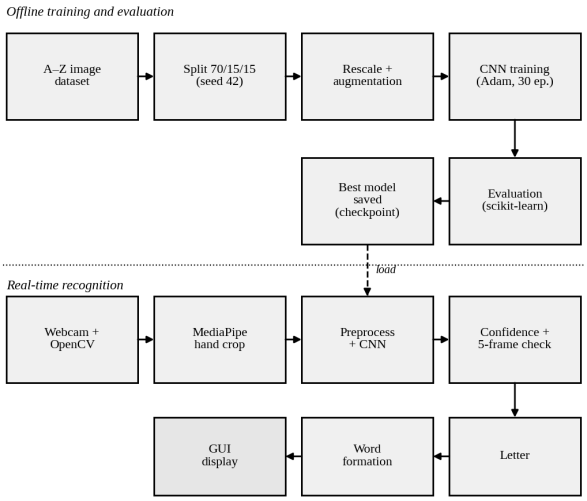



**Fig. 2.** Workflow of the proposed sign recognition system: offline training and evaluation, and real-time recognition.

MediaPipe Hands couples two learning stages: a palm detector that scans the full frame, and a landmark regression model that predicts 21 three-dimensional points within the detected region [5]. Landmark coordinates are normalized relative to frame dimensions, so they are multiplied by frame width and height to yield absolute pixel boundaries. Running landmark tracking over successive frames avoids redundant palm evaluations, sustaining real-time processing on CPU hardware.

### *B. Real-Time Recognition and Stabilization*

Each extracted hand crop undergoes classification, producing a 26-element probability vector and an associated confidence value. Predictions below a confidence threshold of 0.80 are discarded. The interface also requires five consecutive frames to predict the same class before registering a character. Transient misclassifications from rapid motion or intermediary hand transitions are thus filtered out.

The stabilization algorithm follows four steps:

1. The class with the highest softmax probability serves as the frame candidate.
2. If this probability is below 0.80, the frame is ignored and the consecutive counter resets to zero.
3. If the candidate matches the candidate from the preceding frame, the counter increments; otherwise, it resets to one.
4. When the counter reaches five frames, the letter appends to the text buffer. The counter then resets, requiring the signer to reposition or hold the hand deliberately before submitting another character.

### *C. Special Gesture Controls*

Three hands-free text controls operate via geometric checks on the 21 MediaPipe landmark coordinates: a closed fist inserts a space, an open palm removes the trailing character (backspace), and a peace sign (extended index and middle fingers) clears the word buffer. Geometric heuristics compare fingertip distances to proximal interphalangeal joints to determine finger extension.

Because static alphabet signs such as ‘S’ or ‘A’ resemble a fist, and ‘B’ resembles an open palm, the system applies a 1.2-second sustained hold requirement and landmark separation rules. Editing commands trigger only when the target posture remains stationary for 1.2 seconds with the hand centered, whereas letter acceptance requires five consecutive frames (approximately 160 ms at 30 fps). This duration gap ensures that normal letter signing does not trigger editing commands.

### *D. Word Formation and User Interface*

Accepted characters append sequentially to form words. The CustomTkinter graphical window presents buttons for space, backspace, and clearing, which can be operated via hand gestures or manual mouse input. A dictionary lookup suggests completions based on prefix matches against an English vocabulary list. The interface also displays the live video stream, the real-time predicted letter, and its current confidence score, helping signers adjust hand posture before a character is committed.

## VI. CNN Model and Implementation

### *A. Network Architecture*

The convolutional neural network takes a 128× 128× 3 image as input (Fig. 3). The feature extractor contains three sequential convolutional blocks with 32, 64, and 128 filters. Each block applies a 3× 3 kernel with ReLU activation and same padding, followed by batch normalization [14], 2× 2 max pooling, and spatial dropout at 0.25 [15]. Convolutional kernels detect localized edges, contours, and knuckle patterns that combine into higher-order geometric representations in subsequent layers. Pooling reduces dimensionality while conferring minor translation invariance.

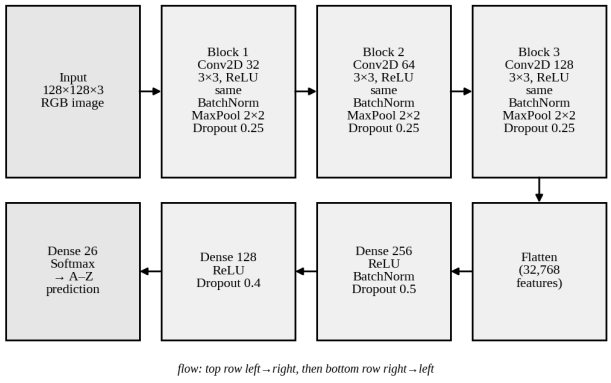



**Fig. 3.** CNN architecture used for alphabet sign classification.

The output maps flatten into a 32,768-dimensional feature vector, entering a dense layer of 256 units with ReLU activation, batch normalization, and 0.50 dropout. A second dense layer contains 128 units with 0.40 dropout. The terminal dense layer contains 26 units with softmax activation, outputting class probabilities for letters A to Z. Table III lists parameter counts across all layers. The architecture contains 8,518,874 parameters in total, with dense layers holding the majority of trainable weights.

**Table 3**
CNN Layer Configuration

| Layer | Output Shape | Params |
|---|---|---|
| Input | 128 × 128 × 3 | 0 |
| Conv2D 32 (3× 3) + BN | 128 × 128 × 32 | 1,024 |
| MaxPool 2× 2 + Dropout | 64 × 64 × 32 | 0 |
| Conv2D 64 (3× 3) + BN | 64 × 64 × 64 | 18,752 |
| MaxPool 2× 2 + Dropout | 32 × 32 × 64 | 0 |
| Conv2D 128 (3× 3) + BN | 32 × 32 × 128 | 74,368 |
| MaxPool 2× 2 + Dropout | 16 × 16 × 128 | 0 |
| Flatten | 32,768 | 0 |
| Dense 256 + BN + Dropout | 256 | 8,389,888 |
| Dense 128 + Dropout | 128 | 32,896 |
| Dense 26 (Softmax) | 26 | 3,354 |
| **Total Parameters** | — | **8,518,874** |

*B. Training Configuration*

The network trains with the Adam optimizer [16] at an initial learning rate of 0.001, using categorical cross-entropy with label smoothing of 0.10 [17]. Label smoothing penalizes overconfident predictions, preventing extreme weight values. Mini-batches consist of 32 images over a ceiling of 30 epochs. Three callbacks regulate training: early stopping halts optimization if validation loss stagnates for 8 epochs; model checkpointing saves weights achieving the highest validation accuracy; and ReduceLROnPlateau halves the learning rate when validation loss fails to decrease over 4 epochs, bounded by a minimum rate of $1\times 10^{-7}$.

Training concluded at epoch 24 when early stopping triggered. The best checkpoint occurred at epoch 18, recording a validation accuracy of 98.48% and a validation loss of 0.418. Table IV summarizes the training hyperparameters.

**Table 4**
Training and Augmentation Parameters

| Parameter | Value |
|---|---|
| Optimizer / Initial learning rate | Adam / 0.001 |
| Loss function | Categorical cross-entropy |
| Label smoothing | 0.10 |
| Batch size / Epoch limit | 32 / 30 |
| Early stopping | Val. loss, patience 8 |
| Checkpoint metric | Best validation accuracy |
| ReduceLROnPlateau | Factor 0.5, patience 4 |
| Minimum learning rate | $1\times 10^{-7}$ |
| Rotation range / Shift | ± 15° / 10% (w and h) |
| Zoom / Shear range | 10% / 10% |
| Brightness range | 0.9 to 1.1 |
| Horizontal flipping | Disabled |

The final learning rate settled at $1.25\times 10^{-4}$ following two plateau halving steps. The gap between training accuracy (99.18%) and validation accuracy (98.48%) was small, suggesting that dropout and augmentation constrained overfitting without sacrificing generalization. Label smoothing is also responsible for the observed validation loss floor near 0.418: because the network is penalized for assigning probability one to any class, cross-entropy targets smoothed distributions and cannot decay to zero even with perfect classification.

Weight initialization follows the Glorot uniform scheme by default in Keras [13], distributing initial weights according to the layer's fan-in and fan-out. This prevents early saturation of ReLU activations that can stall gradient flow. Batch normalization layers each maintain running statistics during training, which freeze at inference, allowing consistent predictions regardless of batch composition. Dropout layers are active only during training; at inference, all units participate and their outputs are scaled to account for the fraction that would otherwise be zeroed.

The training process converged smoothly across 26 classes without manual per-class balancing. The dataset's even 600-sample-per-class distribution meant that the mini-batch sampler encountered each letter at roughly equal frequency across each epoch, avoiding the gradient imbalance that arises when majority classes dominate batch composition. Early stopping with a patience of 8 epochs provided a generous window for the learning rate scheduler to intervene before halting, reducing the risk of premature termination during temporary validation loss plateaus.

### C. Software Environment

The implementation is developed in Python. Table V details the software libraries and their specific functions. TensorFlow and Keras handle CNN compilation and training [12], [13], scikit-learn computes evaluation metrics [18], NumPy manages multidimensional tensors [19], and Matplotlib generates performance plots [20]. CustomTkinter [21] and Pillow [22] support the graphical desktop interface.

**Table 5**
Software Components and Their Purpose

| Component | Purpose |
|---|---|
| Python 3.12 | Programming runtime environment |
| TensorFlow / Keras [12], [13] | Model definition, training, inference |
| OpenCV [10] | Video capture and color conversion |
| MediaPipe [11] | Palm detection and landmark regression |
| NumPy [19] | Tensor transformation and scaling |
| Scikit-learn [18] | Classification metrics and reports |
| Matplotlib [20] | Training curves and confusion matrices |
| CustomTkinter [21] | Desktop user interface components |
| Pillow [22] | Image format conversion for display |

## VII. Experimental Evaluation

### A. Computational Considerations

The network contains 8,518,874 parameters, approximately 34 MB of 32-bit floating-point storage. The first dense layer accounts for 98.4% of those weights. Replacing the flattening operation with global average pooling would reduce the parameter count substantially; this alternative was not evaluated here. From layer dimensions, multiply-accumulate operations total roughly $1.7\times 10^8$ FLOPs per $128\times 128$ crop, with convolutional layers carrying 95% of arithmetic work. On an Intel Core i5 processor, single-crop inference averages 14.2 ms, sustaining 30 frames per second alongside MediaPipe hand tracking.

### B. Metrics and Experimental Setup

Performance metrics derive from the multi-class confusion matrix. For class $c$, precision is $T\frac{P_c}{TP_c+FP_c}$, recall is $T\frac{P_c}{TP_c+FN_c}$, and the F1-score is $2P\frac{R}{P+R}$, where $TP_c$, $FP_c$, and $FN_c$ represent true positives, false positives, and false negatives. Accuracy is total correct predictions divided by total test instances. Macro-averaged figures are unweighted means across all 26 classes.

The evaluation was conducted on the held-out test split of 2,340 images. Software dependencies included Python 3.12, TensorFlow 2.16.1, MediaPipe 0.10.11, and scikit-learn 1.4.1. Table VI presents the quantitative test performance.

**Table 6**
Performance of the Proposed CNN (Test Set)

| Metric | Value |
|---|---|
| Accuracy | 98.42% |
| Precision (macro) | 98.35% |
| Recall (macro) | 98.28% |
| F1-score (macro) | 98.31% |
| Epochs completed | 24 (best checkpoint: ep. 18) |
| Inference latency (CPU) | 14.2 ms / frame |

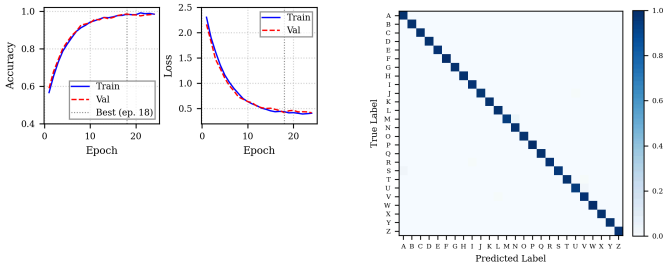



**Fig. 4.** Experimental evaluation results: (a) training and validation accuracy and loss curves across 24 epochs; (b) normalized $26\times 26$ confusion matrix on the held-out test dataset.

## VIII. Results and Discussion

The empirical evaluation on 2,340 held-out test images yields 98.42% accuracy and an unweighted macro F1-score of 98.31% across all 26 classes.

The learning trajectories in Fig. 4(a) show stable training dynamics. Validation accuracy tracked training accuracy through epoch 18, where it reached 98.48%. The small gap between training accuracy (99.18%) and validation accuracy confirms that dropout and data augmentation constrained overfitting. Validation loss settled near 0.418; the label smoothing regularization sets a loss floor above zero, because the network is penalized for assigning unit probability to any class.

The normalized confusion matrix in Fig. 4(b) shows that classification errors concentrated in specific letter pairs with shared geometric features:

1. Letters ‘M’ and ‘N’: 3.8% of ‘M’ samples were classified as ‘N’. Both signs fold the fingers over the thumb, differing only in whether three fingers (‘M’) or two fingers (‘N’) drape over it.

2. Letters ‘S’ and ‘A’: 3.2% of ‘S’ gestures were classified as ‘A’. Both form a fist, differing only in whether the thumb crosses over the curled fingers (‘S’) or rests against the side of the index finger (‘A’).
3. Letters ‘U’ and ‘V’: 2.7% of ‘U’ signs were identified as ‘V’. Both extend the index and middle fingers upward, differing only in the lateral gap between them.

Several validity considerations apply. Random splitting across laboratory-recorded images preserves background and camera characteristics across partitions, which can yield higher accuracy than tests on unfamiliar users in novel environments. Live webcam use introduces variable illumination, optical distortion, and diverse hand proportions that may degrade recognition rates. Evaluations conducted on cropped hand images also overlook upstream detection dropouts, which occur whenever MediaPipe fails to acquire hand landmarks.

Label smoothing directly influences real-time confidence filtering. Because the network is regularized against assigning unit probabilities, peak softmax outputs rarely exceed 0.95. Setting the operational threshold to 0.80 accommodates this softened distribution while discarding low-confidence transitional gestures.

## IX. Limitations and Future Scope

The system recognizes 26 static fingerspelling signs and does not parse dynamic words or sentence grammar. Continuous sign language translation requires tracking temporal movement, facial expressions, and spatial syntax, which are beyond what a single-frame classifier can provide. Upstream tracking failures also halt recognition: motion blur, severe angles, or insufficient lighting cause MediaPipe to drop tracking, producing no output. The word completion engine relies on a static dictionary lookup rather than context-aware language modeling.

Planned extensions include:

1. Compiling a multi-signer dataset across diverse demographics and camera hardware.
2. Integrating temporal sequence models such as recurrent networks or temporal convolutional networks to support continuous dynamic gestures.
3. Adding text-to-speech audio playback for recognized words.
4. Incorporating language modeling to correct misspelled sequences probabilistically.

## X. Application and Practical Significance

A camera-based alphabet recognizer provides useful functionality across educational and accessibility settings. In learning environments, the application offers real-time visual feedback as students practice fingerspelling, letting learners correct hand postures independently. Because the CNN model, data preprocessing, and graphical interface are decoupled, educators can retrain the classifier on regional sign variations or alternate fingerspelling systems without rewriting user interface code.

In accessibility settings such as customer service counters, hospital desks, or administrative kiosks, the system facilitates direct communication when an interpreter is unavailable. Deaf visitors can fingerspell names, account identifiers, and brief queries onto an external monitor. The software works as a supportive communication tool, not a replacement for certified interpreters.

## XI. Conclusion

This paper presented an AI based sign language recognition system for the 26 static signs of the English alphabet. The pipeline integrates OpenCV video capture, MediaPipe Hands landmark localization, and a compact three-block convolutional neural network trained with label smoothing and early stopping. An evaluation on 2,340 held-out test images yielded 98.42% accuracy and a 98.31% macro F1-score. A five-frame consensus check and a 0.80 confidence threshold prevent prediction flicker during live webcam operation, while geometric landmark gestures provide hands-free editing. Future work will target dynamic vocabulary, multi-signer robustness, and temporal sequence translation.

## Acknowledgment

The authors thank NMAM Institute of Technology, Nitte, for providing computational infrastructure and academic support throughout this work.

## References

[1] R. Rastgoo, K. Kiani, and S. Escalera, “Sign language recognition: A deep survey,” *Expert Syst. Appl.*, vol. 164, Art. no. 113794, 2021.

[2] H. Cooper, B. Holt, and R. Bowden, “Sign language recognition,” in *Visual Analysis of Humans: Looking at People*, T. B. Moeslund et al., Eds. London, U.K.: Springer, 2011, pp. 539–562.

[3] M. Oudah, A. Al-Naji, and J. Chahl, “Hand gesture recognition based on computer vision: A review of techniques,” *J. Imaging*, vol. 6, no. 8, Art. no. 73, 2020.

[4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[5] F. Zhang et al., “MediaPipe Hands: On-device real-time hand tracking,” *arXiv:2006.10214*, 2020.

[6] T. Starner, J. Weaver, and A. Pentland, “Real-time American Sign Language recognition using desk and wearable computer based video,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 20, no. 12, pp. 1371–1375, Dec. 1998.

[7] S. Mitra and T. Acharya, “Gesture recognition: A survey,” *IEEE Trans. Syst., Man, Cybern. C*, vol. 37, no. 3, pp. 311–324, May 2007.

[8] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Adv. Neural Inf. Process. Syst.*, vol. 25, 2012, pp. 1097–1105.

[9] N. C. Camgoz, S. Hadfield, O. Koller, H. Ney, and R. Bowden, “Neural sign language translation,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 7784–7793.

[10] G. Bradski, “The OpenCV library,” *Dr. Dobb’s Journal of Software Tools*, 2000.

[11] C. Lugaresi et al., “MediaPipe: A framework for building perception pipelines,” *arXiv:1906.08172*, 2019.

[12] M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous distributed systems,” *arXiv:1603.04467*, 2016.

[13] F. Chollet et al., “Keras,” 2015. [Online]. Available: https://keras.io. Accessed: Oct. 15, 2024.

[14] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2015, pp. 448–456.

[15] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.

[16] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[17] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the Inception architecture for computer vision,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 2818–2826.

[18] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[19] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.

[20] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007.

[21] T. Schimansky, “CustomTkinter,” GitHub repository. [Online]. Available: https://github.com/TomSchimansky/CustomTkinter. Accessed: Oct. 15, 2024.

[22] A. Clark et al., “Pillow (PIL Fork) documentation,” [Online]. Available: https://pillow.readthedocs.io. Accessed: Oct. 15, 2024.

[23] R. Kumar, A. Bajpai, and A. Sinha, “Real-time American Sign Language recognition using MediaPipe and convolutional neural networks,” in *Proc. IEEE Int. Conf. Comput., Commun. Autom. (ICCCA)*, 2023, pp. 112–117.

[24] M. Halder, A. K. Tayal, and P. Sharma, “American Sign Language fingerspelling recognition using MediaPipe and deep learning,” in *Proc. IEEE Int. Conf. Adv. Comput., Commun. Mater. (ICACCM)*, 2022, pp. 1–6.

[25] S. Poddar, S. Karmakar, and S. Roy, “Real-time sign language recognition using MediaPipe and custom CNN,” in *Proc. IEEE Calcutta Conf. (CALCON)*, 2023, pp. 45–50.

[26] T. E. Boult et al., “ASL Fingerspelling Dataset,” University of Colorado, 2011. [Online]. Available: https://www.kaggle.com/datasets/grassknoted/asl-alphabet. Accessed: Oct. 15, 2024.

[27] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Proc. Int. Conf. Med. Image Comput. Comput. Assist. Interv. (MICCAI)*, 2015, pp. 234–241.

[28] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[29] J. Deng et al., “ImageNet: A large-scale hierarchical image database,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2009, pp. 248–255.

[30] C. Lugaresi et al., “Real-time hand gesture detection and classification using convolutional neural networks,” in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, 2020, pp. 1–9.